```
        <li>@item.Key: @Request.Form[item.Key]</li>     ❷
    }
}
</ul>
```

❶ You must check the content type of the request prior to accessing Request.Form to prevent an InvalidOperationException being raised.

❷ Render the name of the collection keys and their associated values.

If you include this code in your existing page, after you have changed the form's method back to `post`, you should see that an item with the key `__RequestVerificationToken` is also included in the output. It is included as part of a security measure to combat cross-site request forgery attacks. I discuss this in greater detail in chapter 13.

The code that iterates `Request.Form` is wrapped in a check to establish the content type of the request. Specifically, the `HasFormContentType` property of the `Request` object is referenced. This returns `true` if the request was performed using the `POST` method. If you don't perform this check outside of an `OnPost` handler, the framework raises an `InvalidOperationException` with the following message: `Incorrect Content-Type`.

5.2 Model binding

Rather than delving into the depths of the form or query collections, the recommended way to work with user input within a `PageModel` class is leveraging the model-binding framework built into ASP.NET Core. Model binding is the process that takes user input from various parts of an HTTP request (the source) and attempts to bind it to C# code (the target). The model-binding framework considers one or more of the following ways users can interact with the application as a source: